

From Heuristic Recall to Explicit Activation: An Evolution Plan for LuckyHarness Memory

Author: LuckyHarness Engineering Notes **Date:** 2026-06-06 **Repository:** luckyharness

Abstract

LuckyHarness currently implements a durable memory system backed by an Obsidian-compatible Markdown vault. Its recall path already contains the core ingredients of an activation system: lexical matching, tier-aware memory weights, time decay, access reinforcement, and graph-based score propagation through wikilinks, aliases, backlinks, and tags. However, these mechanisms are distributed across several functions and are not represented as a first-class activation model.

This paper proposes an incremental evolution of the existing memory recall system into an explicit, explainable, testable activation pipeline. The proposed design introduces a dedicated activation layer, decomposes scores into named components, upgrades graph propagation into path-aware activation spread, unifies `Search` and `SearchParallel`, and exposes debug traces for future tuning. The goal is to preserve existing behavior where possible while making memory retrieval more interpretable and more suitable as a foundation for Graph RAG and agentic context planning.

1. Background

LuckyHarness memory is organized around three tiers:

- short-term memory for recent and transient conversational traces,
- medium-term memory for daily or session-level knowledge,
- long-term memory for durable user facts, project constraints, and reusable rules.

The persistent source of truth is a Markdown vault under the LuckyHarness home directory. Each memory entry stores content, category, tier, importance, timestamps, access count, tags, links, aliases, temporal state fields, block identifiers, and a vault-relative path.

The current system behaves like a lightweight activation network even though it does not name the concept directly. A query activates matching memories, active memories spread partial score to graph-neighbor memories, selected memories are injected into the agent context, and recalled memories receive access reinforcement.

2. Existing Activation-Like Mechanism

The current recall score can be described as:

```
activation_score = match_score * memory_weight
```

where:

```
memory_weight = importance * time_factor * access_bonus
```

2.1 Importance

Importance is a per-entry scalar between 0 and 1. Long-term memories usually receive higher importance, while automatically saved conversational snippets receive lower importance.

2.2 Time Decay

Each tier has a different half-life:

```
short-term: 1 hour  
medium-term: 7 days  
long-term: 365 days
```

The code comments describe the mechanism as exponential decay, but the current implementation is closer to reciprocal decay:

```
time_factor = 1 / (1 + decay)
```

This distinction matters because exponential half-life behavior is easier to reason about, tune, and document.

2.3 Access Reinforcement

Every successful recall increments `AccessCount`. Access count increases future weight by 5 percent per recall, capped at 2x:

```
access_bonus = min(1 + access_count * 0.05, 2.0)
```

This gives the system a simple Hebbian-like reinforcement behavior: memories that are used more often become easier to activate.

2.4 Lexical And Metadata Matching

The existing matcher considers:

- exact or partial content match,
- category match,
- query term matches,

- tags,
- aliases,
- wikilinks.

For Chinese text, query terms are expanded with short n-grams, which improves recall for queries that do not contain whitespace.

2.5 Graph Score Propagation

After direct matches are scored, the memory graph propagates score through:

- forward wikilinks,
- backlinks,
- aliases,
- shared tags.

Current propagation coefficients include:

```
wikilink target: 0.55  
backlink: 0.35  
alias backlink: 0.45  
shared tag: 0.18
```

The propagation formula is approximately:

```
boost = source_score * coefficient * target_weight
```

This is already a small graph activation system. The limitation is that it is not path-aware and does not expose why a memory was boosted.

3. Limitations

3.1 Hidden Score Components

The current recall path returns a final score internally, but it does not expose a structured explanation. This makes tuning difficult. When a memory is over-activated or under-activated, there is no easy way to determine whether the cause was lexical matching, alias matching, graph propagation, recency, importance, or access reinforcement.

3.2 Inconsistent Search Paths

`Search` performs graph propagation and updates access counts. `SearchParallel` searches tiers concurrently but does not perform the same graph propagation or access updates. This creates two subtly different recall semantics.

3.3 Time Decay Semantics

The implementation and comments disagree: comments describe exponential decay, but the code uses reciprocal-style decay. This makes the model harder to reason about and tune.

3.4 Graph Propagation Without Evidence Paths

The existing graph propagation boosts scores but does not record paths. A related memory can be activated by wikilink, backlink, alias, or tag, but the caller cannot inspect which relationship caused the activation.

3.5 Automatic Conversation Noise

LuckyHarness automatically stores user and assistant snippets as short-term memories. These entries are useful for continuity but can become noisy. Context ranking later penalizes `User:` and `Assistant:` snippets, but retrieval-stage activation can still include them.

4. Proposed Evolution

The central proposal is to promote activation into a first-class subsystem.

Add:

```
internal/memory/activation.go
```

The new subsystem should compute activation scores, explain their components, and provide a single entry point for memory recall.

5. Activation Data Model

```
type ActivationScore struct {
    EntryID    string
    Score      float64
    Components ActivationComponents
    Paths      []ActivationPath
}

type ActivationComponents struct {
    Lexical      float64
    Category     float64
    Tags         float64
    Aliases      float64
    Links        float64
    Importance   float64
    Tier         float64
    Recency      float64
    Access       float64
    GraphBoost   float64
}
```

```
type ActivationPath struct {
    FromID string
    ToID   string
    Via    string
    Kind   string
    Weight float64
}
```

This structure makes activation inspectable. A caller can understand not only which memory won, but why it won.

6. Activation Pipeline

The new pipeline should be:

```
query
  -> query normalization
  -> lexical activation
  -> metadata activation
  -> tier, importance, recency, access modifiers
  -> graph activation spread
  -> temporal resolution
  -> context ranking
```

The public entry point should be:

```
func (s *Store) Activate(query string, opts ActivationOptions) []ActivationScore
```

where:

```
type ActivationOptions struct {
    Limit           int
    IncludeGraph    bool
    MaxGraphDepth   int
    MaxGraphBoost   float64
    UpdateAccessStats bool
    Explain         bool
}
```

7. Matching Refactor

The current `memoryMatchScore` returns a single float. It should be refactored into a component-producing function:

```
func matchActivation(e *Entry, query string, terms []string) ActivationComponents
```

The existing scoring constants can be preserved initially:

- content exact match,

- content contains query,
- category contains query,
- query term hits,
- tag hits,
- alias hits,
- wikilink hits.

Preserving constants during the first refactor reduces behavioral risk. Later iterations can tune weights using tests and retrieval logs.

8. Time Decay Upgrade

The time decay model should use true half-life semantics:

```
recency = pow(0.5, age_hours / half_life_hours)
```

Suggested half-lives remain:

```
short-term: 1 hour  
medium-term: 7 days  
long-term: 365 days
```

This gives clear semantics: after one half-life, the recency contribution is halved.

9. Access Reinforcement Upgrade

The current linear access boost can make repeatedly recalled entries too sticky. A logarithmic curve is safer:

```
access_boost = 1 + min(log1p(access_count) * 0.12, 0.75)
```

This preserves reinforcement while reducing runaway effects.

10. Path-Aware Graph Activation

Graph activation should move from simple one-hop score addition to path-aware propagation.

Initial propagation can still be shallow:

```
max_depth: 2  
max_graph_boost: 0.45  
visited set: enabled
```

Relationships should preserve evidence paths:

```
A --wikiLink--> B
A --backlink--> C
A --shared_tag:project--> D
A --alias_backlink--> E
```

The score boost can still use the current idea:

```
boost = source_score * edge_coefficient * target_weight
```

but it should store:

- source memory ID,
- target memory ID,
- relationship type,
- relationship label,
- coefficient,
- path depth.

11. Unified Search Semantics

`Search`, `SearchParallel`, and `Route` should eventually share the same activation core.

Recommended structure:

```
func (s *Store) Activate(query string, opts ActivationOptions) []ActivationScore
func (s *Store) Search(query string) []Entry
func (s *Store) SearchParallel(query string, limit int) []Entry
func (s *Store) Route(query string) RouteAnalysis
```

`Search` becomes a thin wrapper around `Activate`. `SearchParallel` either becomes a concurrent implementation of activation or is deprecated if the unified activation path is fast enough.

12. Context Ranking Integration

Context planning currently applies an additional rank:

```
long memory: +300
medium memory: +200
short memory: +100
health/rule/identity/preference/location/project/plan: +40
conversation: -25
User:/Assistant: -30
importance * 20
```

This should be preserved but integrated with activation:

```
context_score =
  activation_score * 0.65 +
  policy_priority * 0.25 +
  content_quality * 0.10
```

The policy layer should still strongly prefer stable rules, identities, preferences, health facts, project constraints, and location memories over raw automatic conversation snippets.

13. Debug And Evaluation Interface

The system should expose debug output for recall:

```
/recall --debug "telegram gateway"
```

Example output:

```
score 0.83 memory mem_123
- lexical: 0.31
- alias: 0.20
- graph: 0.18 via [[Telegram Gateway]]
- recency: 0.91
- access: 1.12
- tier: long
```

This should not be injected into normal user-facing answers unless requested. It is primarily for developers and retrieval tuning.

14. Implementation Plan

Phase 1: Explainable Activation Without Behavior Drift

- Add `activation.go`.
- Add `ActivationScore`, `ActivationComponents`, `ActivationPath`, and `ActivationOptions`.
- Refactor matching into component form.
- Keep existing scoring constants.
- Add tests proving that previous top results remain stable for representative queries.

Phase 2: Unified Recall Path

- Make `Search` use `Activate`.
- Make `Route` use activated entries.
- Decide whether `SearchParallel` should call `Activate` or be deprecated.
- Ensure access count and accessed time are updated consistently.

Phase 3: Decay And Access Improvements

- Replace reciprocal decay with true half-life decay.
- Replace linear access boost with logarithmic access boost.
- Add regression tests for young, old, frequently accessed, and long-term memories.

Phase 4: Path-Aware Graph Spread

- Replace opaque graph propagation with path-aware propagation.
- Add max depth, graph boost cap, and cycle protection.
- Add tests for wikilink, backlink, alias, and tag propagation.

Phase 5: Debugging And Tuning Tools

- Add debug formatter.
- Extend CLI recall or tool recall with optional explain mode.
- Add retrieval snapshots for common memory queries.
- Track top activated entries and component distributions in tests or diagnostics.

15. Evaluation Metrics

The upgraded system should be evaluated with:

- top-k stability against existing recall results,
- precision of long-term rule recall,
- suppression rate of automatic conversation noise,
- graph expansion usefulness,
- access reinforcement saturation behavior,
- temporal correctness for superseded, expired, and future-dated memories,
- explainability coverage: every returned score should have at least one meaningful component or path.

16. Expected Benefits

The proposed activation layer should provide:

- clearer memory retrieval behavior,
- easier tuning,
- more reliable context injection,
- reusable scoring infrastructure for Graph RAG,
- better debugging for unexpected recall,
- a cleaner separation between memory storage and memory activation.

17. Conclusion

LuckyHarness already contains the foundation of a memory activation system. Importance, time decay, access reinforcement, metadata matching, and graph propagation are all present. The next step is to make this implicit mechanism explicit.

By introducing a first-class activation layer, LuckyHarness can evolve from heuristic recall to explainable activation. This preserves the pragmatic strengths of the current system while preparing the architecture for deeper graph-based retrieval and more reliable agent memory behavior.